Unit-2

BOOLEAN ALGEBRA AND MINIMIZATION TECHNIQUES

Dr. V S M Srinivasavarma,
Assistant Professor,
Computer Science and Engineering,
Shiv Nadar University Chennai.

Boolean Algebra

- Boolean algebra is a mathematical system for the manipulation of variables that can have one of two values.
  - In formal logic, these values are “true” and “false.”
  - In digital systems, these values are “on” and “off,” 1 and 0, or “high” and “low.”
- Boolean expressions are created by performing operations on Boolean variables.
  - Common Boolean operators include AND, OR, and NOT.

Boolean Algebra

- A Boolean operator can be completely described using a truth table.
- The truth table for the Boolean operators AND and OR are shown at the right.
- The AND operator is also known as a Boolean product. The OR operator is the Boolean sum.

X AND Y

X	Y	XY
0	0	0
0	1	0
1	0	0
1	1	1

X OR Y

X	Y	X+Y
0	0	0
0	1	1
1	0	1
1	1	1

Boolean Algebra

- The truth table for the Boolean NOT operator is shown at the right.
- The NOT operation is most often designated by an overbar. It is sometimes indicated by a prime mark (‘) or an “elbow” (¬).

NOT x

x	$\overline{x}$
0	1
1	0

Boolean Algebra

- A Boolean function has:
  - At least one Boolean variable,
  - At least one Boolean operator, and
  - At least one input from the set {0,1}.
- It produces an output that is also a member of the set {0,1}.

Now you know why the binary numbering system is so handy in digital systems.

Boolean Algebra

- The truth table for the Boolean function:

$$F(x,y,z) = x\bar{z}+y$$

is shown at the right.

- To make evaluation of the Boolean function easier, the truth table contains extra (shaded) columns to hold evaluations of subparts of the function.

$$F(x,y,z) = x\bar{z}+y$$

x	y	z	$\bar{z}$	$x\bar{z}$	$x\bar{z}+y$
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	0	0	1

Boolean Algebra: Basic Theorems

$$X + 0 = X \qquad X \cdot 1 = X$$

$$X + 1 = 1 \qquad X \cdot 0 = 0$$

$$X + X = X \qquad X \cdot X = X$$

$$(X')' = X \qquad X + XY = X$$

$$X + X' = 1 \qquad X(X + Y) = X$$

Simplification Theorems

1. $XY + XY' = X$
2. $(X + Y)(X + Y') = X$
3. $X + XY = X$
4. $X(X + Y) = X$
5. $(X + Y')Y = XY$
6. $XY' + Y = X + Y$

Simplification Theorems

- Proof 6.

$$
\begin{aligned}
\text{R.H.S.} &= X+Y \\
&= X(Y+Y')+Y(X+X') \\
&= XY+XY'+XY+X'Y \\
&= XY+XY'+X'Y \\
&= (X+X')Y+XY' \\
&= Y+XY' \\
&= \text{L.H.S.}
\end{aligned}
$$

Operator Precedence

- Parenthesis
- NOT
- AND
- OR

Just recall your operator precedence in the Arithmetic (BODMAS).

Inversion

$$(X+Y)' = X'Y'$$

$$(XY)' = X' + Y'$$

$$(X_1 + X_2 + \ldots + X_n)' = X_1' X_2' \ldots X_n'$$

$$(X_1 X_2 \ldots X_n)' = X_1' + X_2' + \ldots + X_n'$$

Prove with the truth tables…

- The complement of the product is the sum of the complements
- The complement of the sum is the product of the complements

Examples

Find the complements of

$$[(A' + B)C']' = ?$$

$$[(AB' + C)D' + E]' = ?$$

$$\left[A + ((BC')' + D)'\right]'' = ?$$

Boolean Algebra

- Most Boolean identities have an AND (product) form as well as an OR (sum) form. We give our identities using both forms. Our first group is rather intuitive:

Identity Name	AND Form	OR Form
Identity Law	$1x = x$	$0 + x = x$
Null Law	$0x = 0$	$1 + x = 1$
Idempotent Law	$xx = x$	$x + x = x$
Inverse Law	$x\bar{x} = 0$	$x + \bar{x} = 1$

Boolean Algebra

- Our second group of Boolean identities should be familiar to you from your study of algebra:

Identity Name	AND Form	OR Form
Commutative Law	$xy = yx$	$x+y = y+x$
Associative Law	$(xy)z = x(yz)$	$(x+y)+z = x + (y+z)$
Distributive Law	$x+yz = (x+y)(x+z)$	$x(y+z) = xy+xz$

Duality

- The dual of a Boolean expression is obtained by interchanging Boolean sums and Boolean products and interchanging 0s and 1s.

Examples:

The dual of $\mathbf{x(y + z)}$ is $\mathbf{x + yz.}$

The dual of $\mathbf{-x \cdot 1 + (-y + z)}$ is $\mathbf{(-x + 0)((-y)z).}$

minterms

- Consider variables *A* and *B*
- Assume that they are somehow combined with AND operator
- There are 4 possible combinations

$$AB, \quad A'B, \quad AB', \quad A'B'$$

- Each of those terms is called a minterm (standard product)
- In general, if there are n variables, there are 2^n minterms

Exercise

- List the **minterms** for 3 variables

A	B	C	maxterm	Designation
0	0	0	A ' B ' C'	m_0
0	0	1	A ' B ' C	m_1
0	1	0	A ' B C'	m_2
0	1	1	A ' B C	m_3
1	0	0	A B ' C'	m_4
1	0	1	A B ' C	m_5
1	1	0	A B C'	m_6
1	1	1	A B C	m_7

Maxterms

- Consider variables *A* and *B*
- Assume that they are somehow combined with OR operator
- There are 4 possible combinations

$$A+B, \quad A^{'}+B, \quad A+B^{'}, \quad A^{'}+B^{'}$$

- Each of those terms is called a maxterm (standard sums)
- In general, if there are n variables, there are 2^n maxterms

Exercise

- List the maxterm for 3 variables

A	B	C	Maxterm	Designation
0	0	0	A+B+C	M_0
0	0	1	A+B+C'	M_1
0	1	0	A+B'+C	M_2
0	1	1	A+B'+C'	M_3
1	0	0	A'+B+C	M_4
1	0	1	A'+B+C'	M_5
1	1	0	A'+B'+C	M_6
1	1	1	A'+B'+C'	M_7

Example

- Express *F* in the sum of minterms and product of maxterms formats

$$F = A + BC'$$

$$\begin{aligned}
F &= A + BC' = A(B+B')(C+C') + (A+A')BC' \\
&= ABC + ABC' + AB'C + AB'C' + ABC' + A'BC' \\
&= ABC + ABC' + AB'C + AB'C' + A'BC' \\
&= m_7 + m_6 + m_5 + m_4 + m_2 \\
&= \sum(2,4,5,6,7) \\
&= \prod(0,1,3)
\end{aligned}$$

Sum-of-Products

- All products are the product of single variable only

$$AB' + CD'E + AC'E' \leftarrow \text{YES}$$

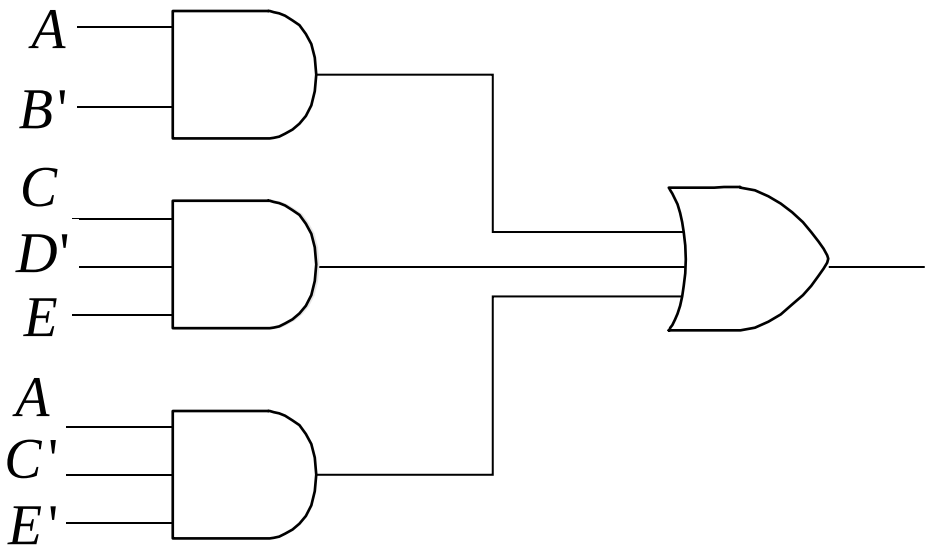
$$A + B' + C + D'E \leftarrow \text{YES}$$

$$(A + B)CD + EF \leftarrow \text{NO}$$

Sum-of-Products

$$AB' + CD'E + AC'E'$$

One or more AND gates feeding a single OR gate at the output



Product-of-Sums

All sums are the sums of single variables

$$(A+B')(C+D'+E)(A+C'+E') \leftarrow \text{YES}$$

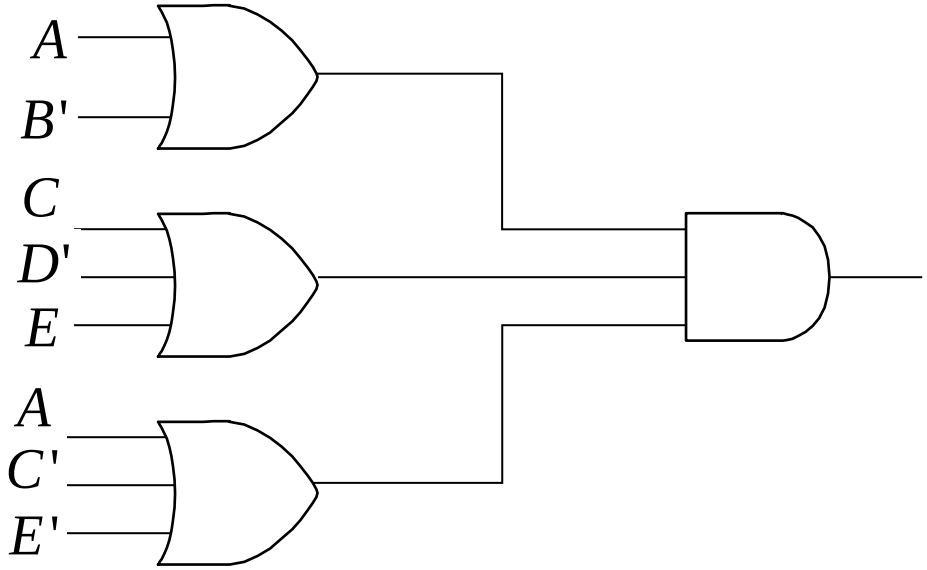
$$AB'C(D'+E) \leftarrow \text{YES}$$

$$(A+B)(C+D)+EF \leftarrow \text{NO}$$

Product-of-Sums

One or more OR gates feeding a single AND gate at the output

$$(A + B')(C + D' + E)(A + C' + E')$$



Simplifications of Boolean Equations

- Every Boolean equation represent some logic function which has some computational relevance.
- Every Boolean logic has to be implemented by using logic gates
- For time being, let us assume, we need to implement them using Basic Gates: AND, OR, and NOT.
- The objective is to implement a given Boolean function using minimum number of Gates.
- Why minimum Gates.? To reduce the cost and energy consumption of the chip.

Canonical & Standard Forms

We can express a Boolean equation in following two ways.

- Canonical SoP form
- Canonical PoS form

- F = A + B’C
- F = A’B’C + AB’C’ + AB’C + ABC’ + ABC ➔ Canonical SoP form
- F = Sigma(1, 4, 5, 6, 7)
- A given function in canonical SoP form can be represented in the canonical PoS form as follows:
  - F = Pi(0, 2, 3)
  - F = (A+B+C)*(A+B’+C)*(A+B’+C’) ➔ Canonical PoS form

- Both canonical SoP and canonical PoS forms are **Dual** to each other. Functionally, these two forms are same. Based on the requirement, we can use one of these two forms.

Canonical & Standard Forms

Whereas the reduced form is also referred to as the standard form

- F = A’B’C + AB’C’ + AB’C + ABC’ + ABC ➔ Canonical SoP form
- F = A + B’C ➔ Standard SoP form

- F = (A+B+C)*(A+B’+C)*(A+B’+C’) ➔ Canonical PoS form
- F = Pi(0, 2, 3)
- F = (A=B’)*(A+C) We can use the K-map for obtaining the Standard PoS form from the canonical PoS form.

Simplifications of Boolean Equations

Any Boolean function can be reduced by using one of the following techniques:

1. Naïve Approach: Using Boolean Laws
2. Karnaugh Map (K-Map)
3. Quine-Mccluskey Method (Tabular Approach)

Naïve Approach: Using Boolean Laws

$F = \bar{A}\bar{B}C + \bar{A}\mathrm{B}\,\bar{C} + \mathrm{A}\,\bar{B}\,\bar{C} + \mathrm{ABC}$

$F = \bar{A}(\bar{B}C + \mathrm{B}\,\bar{C}) + \mathrm{A}(\bar{B}\,\bar{C} + \mathrm{BC})$

$F = \bar{A}(B \oplus C) + \mathrm{A}(\mathrm{B} \odot \mathrm{C})$

$F = \bar{A}(B \oplus C) + \mathrm{A}(\overline{B \oplus C})$

$F = A \oplus B \oplus C$

Here, F represents the sum output of a Full-adder circuit. More about a Full-adder will be discussed in Unit -3.

Ex-Or: $\oplus$. $\mathrm{A} \oplus \mathrm{B} = \bar{A}B + \mathrm{A}\,\bar{B}$

Ex-Nor: $\odot$. $\mathrm{A} \odot \mathrm{B} = \bar{A}\bar{B} + \mathrm{AB}$

Recall that, $\overline{(A + B)} = \bar{A} * \bar{B}$

$$
\begin{aligned}
(\overline{\mathrm{A} \oplus \mathrm{B}}) &= \overline{\bar{A}B + \mathrm{A}\,\bar{B}} \\
&= (\overline{\bar{A}B})*(\overline{\mathrm{A}\,\bar{B}}) \\
&= (\bar{\bar{A}} + \bar{B})*(\bar{A} + \bar{\bar{B}}) \\
&= (\mathrm{A} + \bar{B})*(\bar{A} + \mathrm{B}) \\
&= (\mathrm{A}\,\bar{A} + \mathrm{AB} + \bar{A}\bar{B} + \mathrm{B}\,\bar{B}) \\
&= (0 + \mathrm{AB} + \bar{A}\bar{B} + 0) \\
&= (\bar{A}\bar{B} + \mathrm{AB}) \\
&= \mathrm{A} \odot \mathrm{B}
\end{aligned}
$$

Naïve Approach: Using Boolean Laws

Note, while minimizing any Boolean expression, we need to recall the basic Boolean laws and apply them as needed.

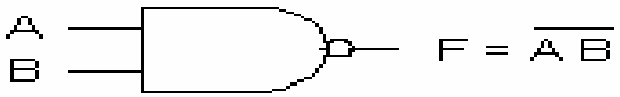
<u>Exercises:</u>

Minimize: 1. $F = \bar{A}BC + A\bar{B}\mathrm{C} + \mathrm{AB}\bar{C} + \mathrm{ABC}$

2. $\mathrm{F} = \bar{A}\bar{B}C + \bar{A}\mathrm{B}\,\bar{C} + \bar{A}BC + \mathrm{ABC}$

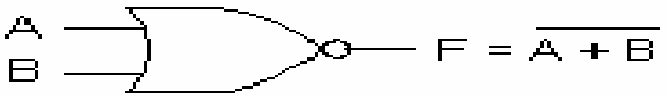
Logic Gates

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0



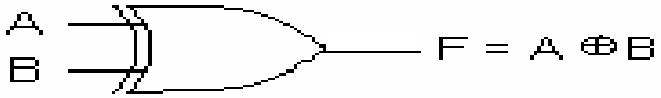
NAND

A	B	F
0	0	1
0	1	0
1	0	0
1	1	0



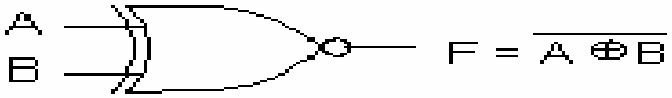
NOR

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0



Exclusive-OR (XOR)

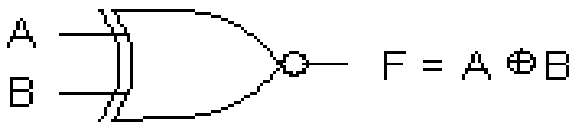
A	B	F
0	0	1
0	1	0
1	0	0
1	1	1



Exclusive-NOR (XNOR)

Exclusive-OR

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0



Exclusive-OR (XOR)

$$A \oplus 0 = A$$

$$A \oplus 1 = A'$$

$$A \oplus A = 0$$

$$A \oplus A' = 1$$

$$A \oplus B = B \oplus A$$

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

$$A(B \oplus C) = AB \oplus AC$$

$$(A \oplus B)' = A \oplus B' = A' \oplus B = AB + A'B'$$

$$A \oplus B = 1 \Leftrightarrow A = 1 \quad \text{or} \quad B = 1 \quad \text{but not both}$$

In other words, the output of Ex-Or gate is high when both the inputs are different.
Ex-Nor is the complement of Ex-Or output.

Logic Gates

- NAND and NOR gates are also called as Universal Gates.
- **AND+NOT = NAND**, **OR+NOT = NOR**.
- In real-world, for VLSI digital circuit implementation, the VLSI library has only two macros in the library (NAND and NOR)
- Any equation should be implemented using these macros
- Why NAND and NOR.? Please explore and answer by next class (+2 Marks will be rewarded in the CIA*).
- And, using NAND and NOR, we can implement all other gates, i.e. AND, OR, NOT, Ex-OR, and Ex-Nor.

Assignment-II: (i) Implement all other gates using NAND gate alone (Cond.: Use minimum no. of gates).

***: Conditions apply.**

Karnaugh Maps (K-Maps)

- A visual way to simplify logic expressions
- It gives the most simplified form of the expression
- Algebra Method becomes more complicated when the number of variables in the Boolean equation exceeds 3 (It is sometimes cumbersome for 3-variables as well)

Motivation:

- Simplification of logic expression using Boolean algebra is awkward because:
  - it lacks specific rules to predict the most suitable next step in the simplification process
  - it is difficult to determine whether the simplest form has been achieved.

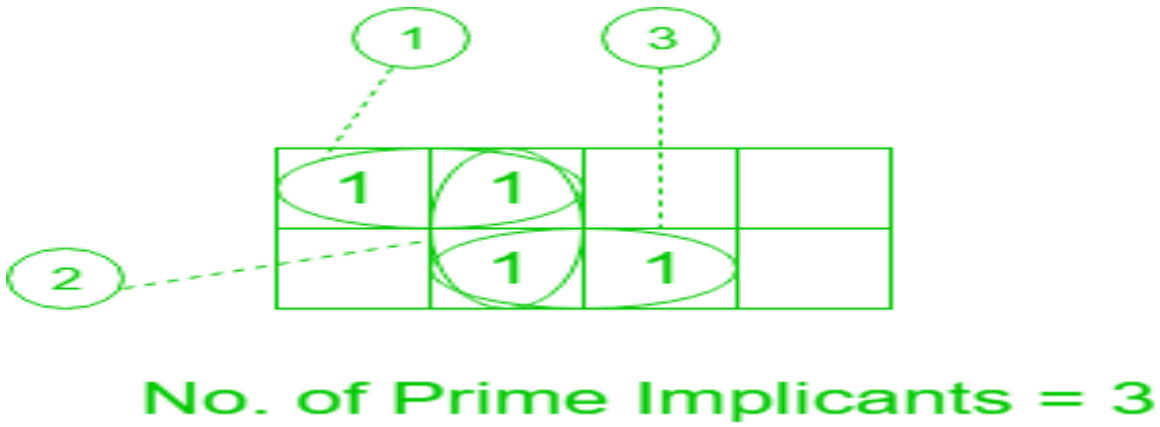
K-Map

- A Karnaugh map is a graphical method used to obtained the most simplified form of an expression in a standard form (Sum-of-Products or Product-of-Sums).
- The simplest form of an expression is the one that has the minimum number of terms with the least number of literals (variables) in each term.
- By simplifying an expression to the one that uses the minimum number of terms, we ensure that the function will be implemented with the minimum number of gates.
- By simplifying an expression to the one that uses the least number of literals for each terms, we ensure that the function will be implemented with gates that have the minimum number of inputs.

K-Map

- **Implicant:** It is a product/minterm term in Sum of Products (SOP) or sum/maxterm term in Product of Sums (POS) of a Boolean function.
    - E.g., consider a boolean function, F = AB + ABC + BC. Implicants are AB, ABC, and BC.
- **Prime Implicants:** A group of squares or rectangles made up of a bunch of adjacent minterms which is allowed by the definition of K-Map are called **prime implicants(PI)** i.e. all possible groups formed in K-Map.

Ex:

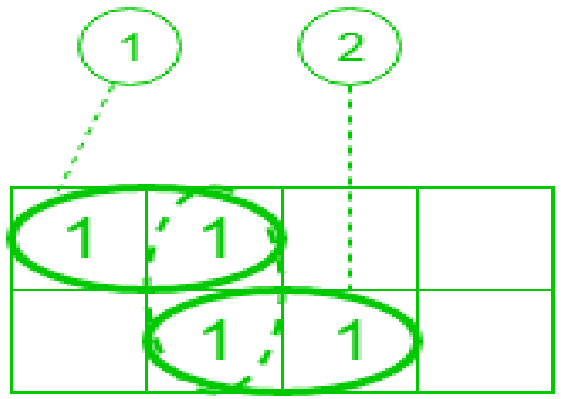


No. of Prime Implicants = 3

K-Map

- **Essential Prime Implicants:** These are those subcubes (groups) that cover at least one minterm that can't be covered by any other prime implicant. **Essential prime implicants(EPI)** are those prime implicants that always appear in the final solution.

Ex:



No. of Essential Prime Implicants = 2

Three-Variable K-Map

$f = \sum(0,4) = \overline{B}\,\overline{C}$

A \ BC	00	01	11	10
0	1	0	0	0
1	1	0	0	0

$f = \sum(4,5) = A\,\overline{B}$

A \ BC	00	01	11	10
0	0	0	0	0
1	1	1	0	0

$f = \sum(0,1,4,5) = \overline{B}$

A \ BC	00	01	11	10
0	1	1	0	0
1	1	1	0	0

$f = \sum(0,1,2,3) = \overline{A}$

A \ BC	00	01	11	10
0	1	1	1	1
1	0	0	0	0

$f = \sum(0,4) = \overline{A}\,C$

A \ BC	00	01	11	10
0	0	1	1	0
1	0	0	0	0

$f = \sum(4,6) = A\,\overline{C}$

A \ BC	00	01	11	10
0	0	0	0	0
1	1	0	0	1

$f = \sum(0,2) = \overline{A}\,\overline{C}$

A \ BC	00	01	11	10
0	1	0	0	1
1	0	0	0	0

$f = \sum(0,2,4,6) = \overline{C}$

A \ BC	00	01	11	10
0	1	0	0	1
1	1	0	0	1

Three-Variable K-Map

A \ BC	00	01	11	10
0		1		
1	1		1	1

A \ BC	00	01	11	10
0	1		1	1
1	1			1

A \ BC	00	01	11	10
0			1	1
1	1	1		

A \ BC	00	01	11	10
0			1	
1	1		1	1

A \ BC	00	01	11	10
0		1	1	1
1		1	1	

A \ BC	00	01	11	10
0				
1				

Four-Variable K-Maps

AB \ CD	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	0	0	0

$$f = \sum(0,8) = \overline{B} \bullet \overline{C} \bullet \overline{D}$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	0	0
11	0	1	0	0
10	0	0	0	0

$$f = \sum(5,13) = B \bullet \overline{C} \bullet D$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	1	1	0
10	0	0	0	0

$$f = \sum(13,15) = A \bullet B \bullet D$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(4,6) = \overline{A} \bullet B \bullet \overline{D}$$

AB \ CD	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(2,3,6,7) = \overline{A} \bullet C$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	1	0	0	1
10	0	0	0	0

$$f = \sum(4,6,12,14) = B \bullet \overline{D}$$

AB \ CD	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	0	0	0	0
10	0	0	1	1

$$f = \sum(2,3,10,11) = \overline{B} \bullet C$$

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

$$f = \sum(0,2,8,10) = \overline{B} \bullet \overline{D}$$

Four-Variable K-Maps

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	0	0	0	0

$f = \sum(4,5,6,7) = \overline{A} \bullet B$

AB \ CD	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	0
10	0	0	1	0

$f = \sum(3,7,11,15) = C \bullet D$

AB \ CD	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

$f = \sum(0,3,5,6,9,10,12,15)$

$f = A \otimes B \otimes C \otimes D$

AB \ CD	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

$f = \sum(1,2,4,7,8,11,13,14)$

$f = A \oplus B \oplus C \oplus D$

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

$f = \sum(1,3,5,7,9,11,13,15)$

$f = D$

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

$f = \sum(0,2,4,6,8,10,12,14)$

$f = \overline{D}$

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	1	1
10	0	0	0	0

$f = \sum(4,5,6,7,12,13,14,15)$

$f = B$

AB \ CD	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	0	0
10	1	1	1	1

$f = \sum(0,1,2,3,8,9,10,11)$

$f = \overline{B}$

Four-Variable K-Maps

AB \ CD	00	01	11	10
00	1	1		1
01	1	1		1
11	1	1		1
10	1	1		

AB \ CD	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

AB \ CD	00	01	11	10
00				
01	1	1	1	
11	1	1		1
10	1			

AB \ CD	00	01	11	10
00		1	1	
01	1	1	1	1
11	1		1	1
10			1	

AB \ CD	00	01	11	10
00				
01				
11				
10				

AB \ CD	00	01	11	10
00				
01				
11				
10				

Design of combinational digital circuits

- Steps to design a combinational digital circuit:
  - From the problem statement derive the truth table
  - From the truth table derive the unsimplified logic expression
  - Simplify the logic expression
  - From the simplified expression draw the logic circuit

Design of combinational digital circuits

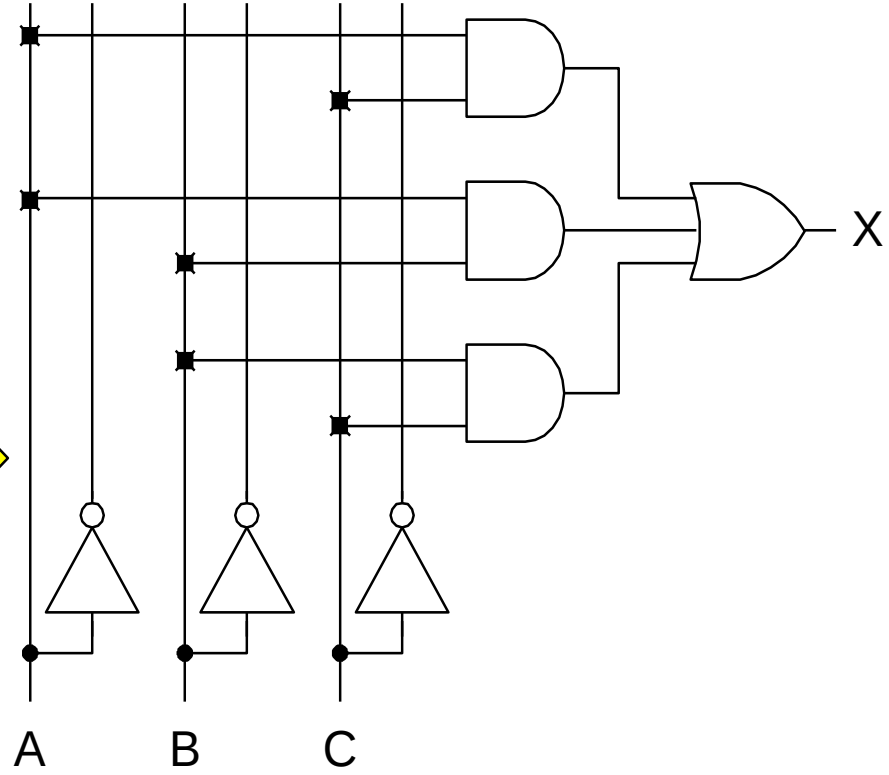
- Example: Design a 3-input (A,B,C) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input has more ones than zeros.

	Inputs			Output
	A	B	C	X
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$$X = \sum(3,5,6,7)$$

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$X = AC + AB + BC$$



Design of combinational digital circuits

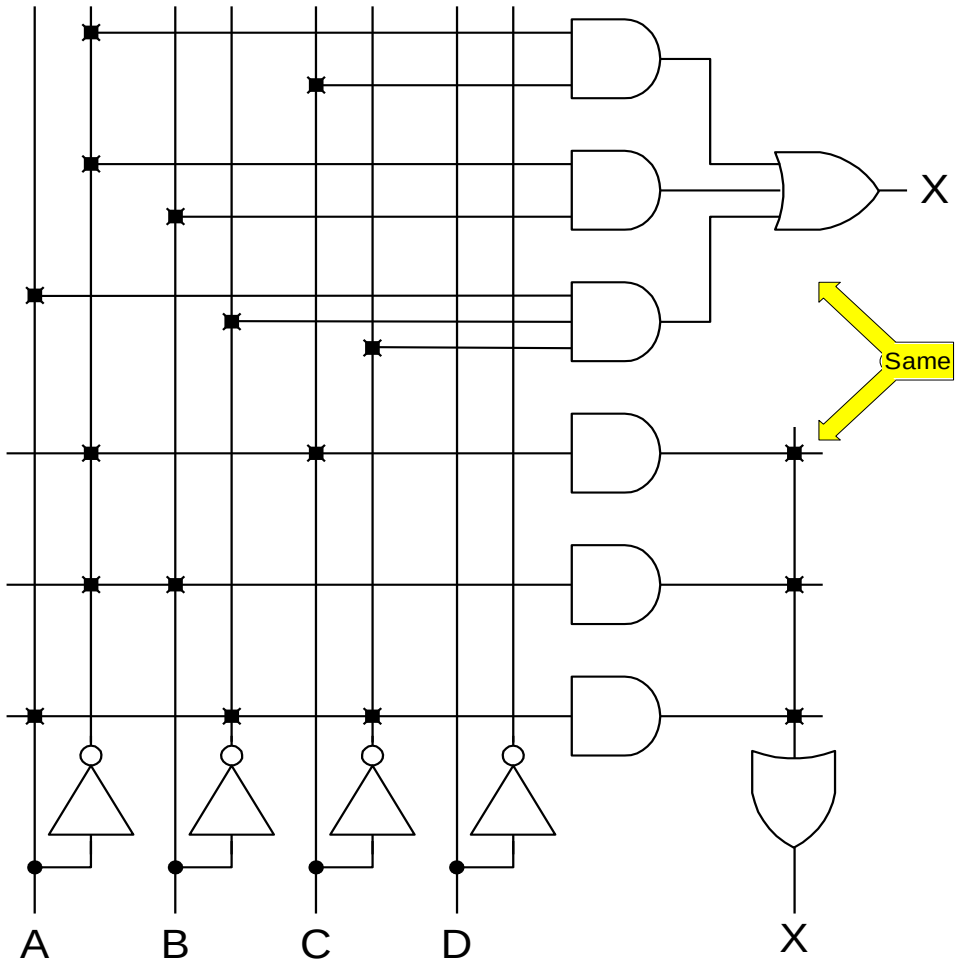
- Example: Design a 4-input (A,B,C,D) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input is between 2 and 9 (including).

	Inputs				Output
	A	B	C	D	X
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	0

$$X = \sum(2,3,4,5,6,7,8,9)$$

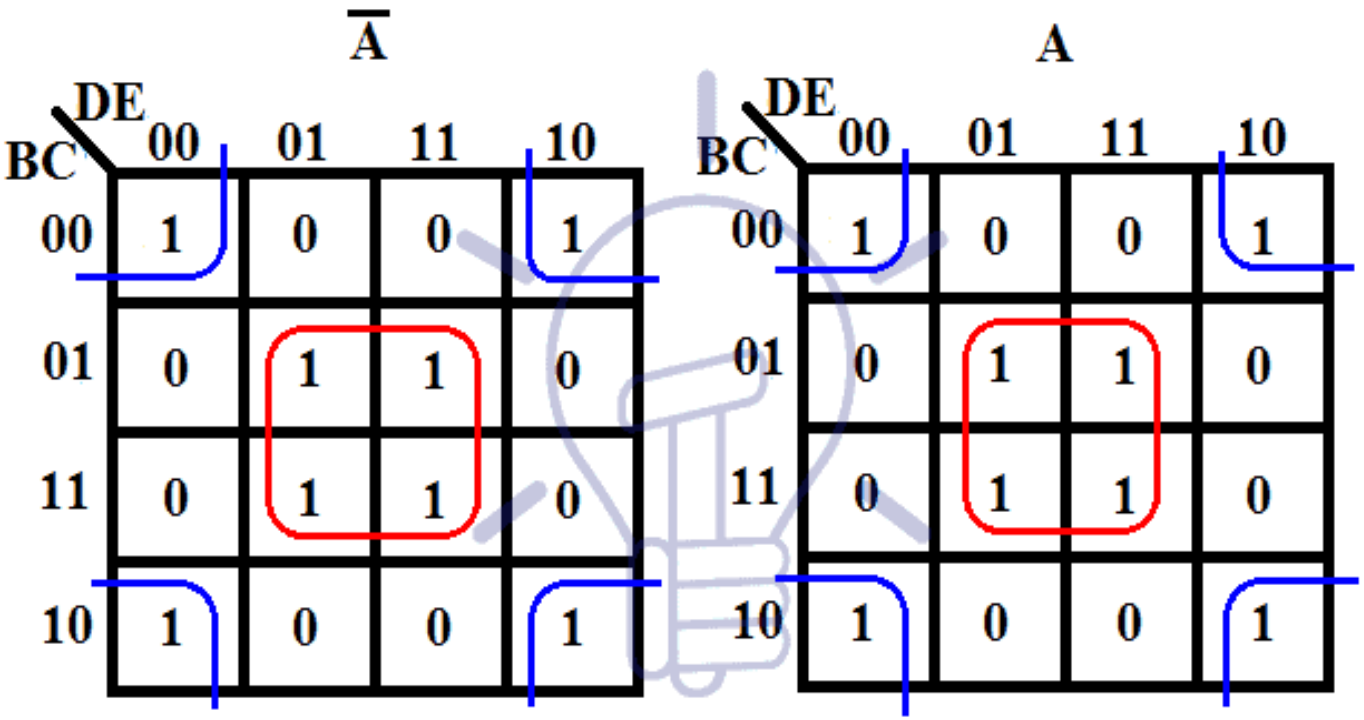
AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	1	1
11	0	0	0	0
10	1	1	0	0

$$X = \overline{A}C + \overline{A}B + A\overline{B}\,\overline{C}$$



Quine-McCluskey Method

- K-Map limitation is that it can be used only till 6-variables.
- Note, for 5-variable itself (refer below figure), the process is little complicated and require careful observation while pairing the minterms.



Groups of 8

Quine-McCluskey Method

- Tabular method is standardly preferred for reducing the Boolean equation having more than 6 variables (Indeed, anything more than 5, Tabular method is preferred)
- Standard Tabular method was improved by Quine and McCluskey to reduce the total steps involved
- Due to time constraints, we will directly jump into Quine-McCluskey Method.

Quine-McCluskey Method

The Tabular Method:

1. **Grouping:** Use the number of "1"s in the minterm to group the minterms. Preserve groups derived from this grouping in adjacent columns.

2. **Comparison:** Each minterm in a group is compared with every other minterm in the adjacent group to find minterms which differ in exactly one variable.

3. **New List:** Constructs a new list with terms of one fewer variables, keeping track of which minterms were covered. This step also should happen in group-wise manner.
   Ex: Terms formed by comparing group-0 and group-1 should be kept as one new group in the new list formed.

Quine-McCluskey Method

The Tabular Method:

4. **Comparison:** Compares every element in a group in the new list with all the elements in the adjacent group of the new list. Find terms that differ in one more variable. This step also should happen in group-wise manner.

5. Repeats Steps 3 and 4 until done.

Q-M on $F(x,y,z)=\sum m(2,3,6,7)$

	Initial	Column (a)	Column (b)	Column(c)
Step 1	**One 1**	**010 m_2**		

	Two 1's	**011 m_3**		
		110 m_6		

	Three 1's	**111 m_7**		

Step 2: Compare terms from adjacent groups.

Group 1 => Group 2

010: (011 $\Rightarrow$ 01-), (110 $\Rightarrow$-10)

Group 2 => Group 3

011: (111$\Rightarrow$-11)

110: (111 $\Rightarrow$ 11-)

Q-M on $F(x,y,z)=\sum m(2,3,6,7)$

Step 3

Initial	Column (a)	Column (b)	Column(c)
One 1	010 m_2√	(01-) m_2,m_3	
	----------	(-10) m_2,m_6	
Two 1's	011 m_3√	----------	
	110 m_6√	(-11) m_3,m_7	
	----------	(11-) m_6,m_7	
Three 1's	111 m_7√		

Step 4: Repeat on Column (b)

Group (1-2) => Group (2-3)

(01-): (-11⇒ Ø), (11- ⇒ -1-)

(-10): (-11 ⇒-1-), (11-⇒Ø)

Q-M on $F(x,y,z)=\sum m(2,3,6,7)$

Step 5

Initial	Column (a)	Column (b)	Column(c)
One 1	010 m_2√	(01-) m_2,m_3√	(-1-)m_2,m_3,m_6,m_7
	----------	(-10) m_2,m_6√	(-1-)m_2,m_3,m_6,m_7
Two 1's	011 m_3√	----------	
	110 m_6√	(-11) m_3,m_7√	
	----------	(11-) m_6,m_7√	
Three	111 m_7√		

Note that the resulting terms are duplicates and are unchecked at termination.

Final Result: F(x,y,z) = y

In general, when no new terms can be generated, the set of all unchecked terms give the result.

Q-M on $F(x,y,z)=\sum m(2,3,6,7)$

- The steps mentioned so far helps us in identifying the prime implicants.
- **Prime Implicants**: 1. All the minterms (Step-1) or minterm combinations (Step-3) which are unticked and the minterms combinations in the last step (Step-5) are the prime implicants.
- Recall that, the minimized Boolean equation should contain only **Essential Prime Implicants (EPIs).**
- So, we need to identify the EPIs. A sample table used for identifying the EPIs from PIs is shown below.
- This procedure can be well explained with the help of an example.
- Note, once the procedure for identifying the EPIs from PIs is completed, you are advised to draw a K-Map for minimizing the Boolean expression (Note, this step is not part of Q-M and this will enhance the understanding of the step). This gives an essential clarity on the final step.

PIs	**m1**	**m2**	**m3**	**m9**	**m13**
PI-1					
PI-2					
PI-3					

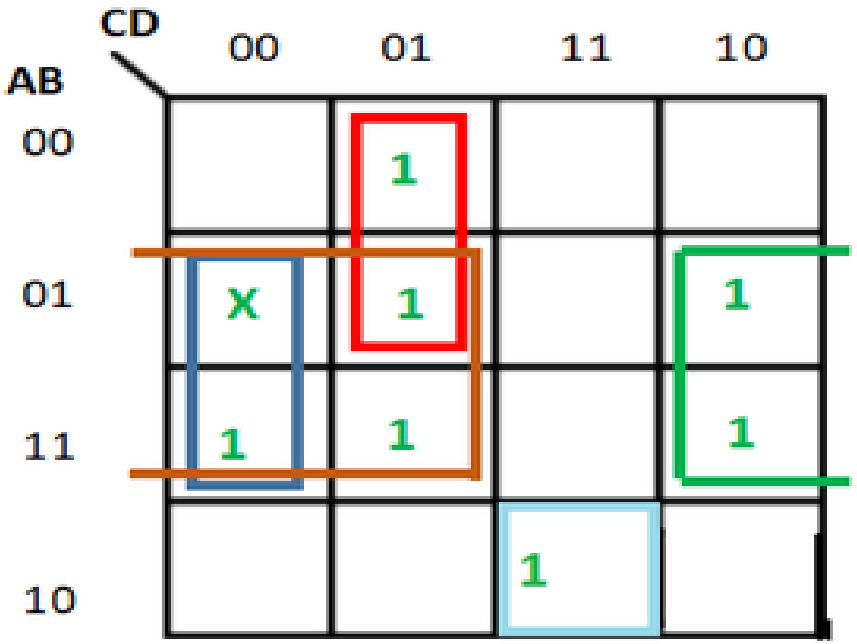
Exercises

- Minimize the following using Q-M Method:

1. F(A, B, C, D) = $\sum m\,(0, 5, 7, 8, 9, 10, 11, 14, 15)$
2. F(A, B, C, D) = $\sum m\,(0, 2, 3, 4, 6, 7, 10, 11, 12, 14, 15)$
3. F(A, B, C, D) = $\sum m\,(1, 3, 4, 5, 9, 10, 11)$+ $\sum d\,(6, 8)$

K-Map with Don’t cares

f = m(1, 5, 6, 11, 12, 13, 14) + d(4)



f = BC' + BD' + A'C'D + AB'CD

K-Map with Don’t cares

Exercises:

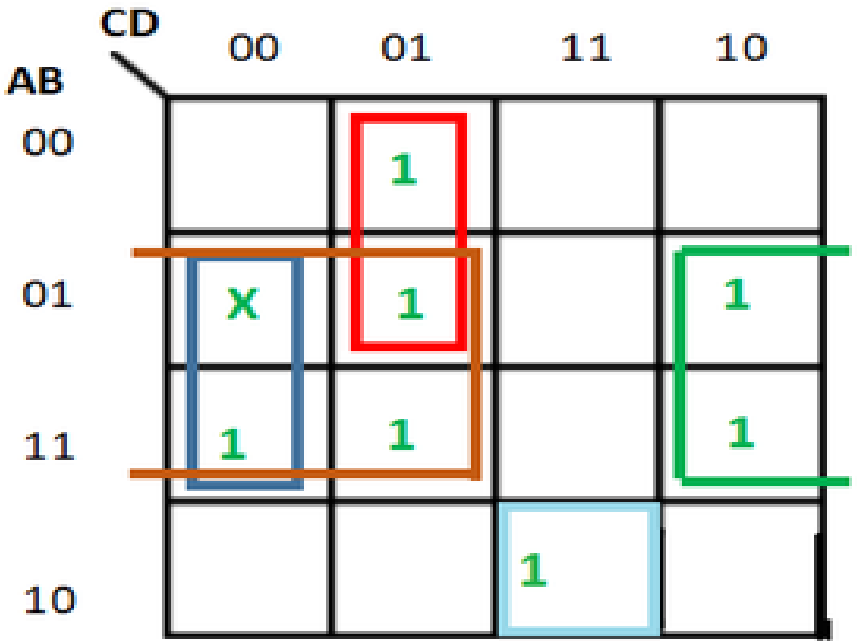
1. F(A, B, C, D) = m(1, 2, 6, 7, 8, 13, 14, 15) + d(0, 3, 5, 12)

2. F(A, B, C, D) = m(0, 1, 2, 3, 4, 5) + d(10, 11, 12, 13, 14, 15)

Note, repeat the above exercises using Q-M method as well.

K-Map with Don’t cares

f = m(1, 5, 6, 11, 12, 13, 14) + d(4)



f = BC' + BD' + A'C'D + AB'CD

Thank You